Academic English Practice App

Agile Development Plan (Revised — Web Application)

Tech Stack: React + Flask + SQLite + OpenAI API

1. Product Goal & Vision

Product Goal: Build a web application that helps incoming university students transition from everyday English to academic English, enabling them to confidently participate in lectures, seminars, and academic discussions before starting their degree.

Target Users: Non-native English speakers at pre-university level, anxious about classroom participation, with limited time (prefer 5–15 min sessions).

2. Technology Stack

Layer	Technology	Why
Frontend	React + Vite + Ant Design	Fast dev build, rich component library, large ecosystem
Backend	Python Flask	Lightweight, easy AI integration, fast to develop
Database	SQLite + SQLAlchemy	Zero config, easy migration to PostgreSQL later
AI Service	OpenAI Chat API	Conversation, quiz generation, feedback
Voice	Web Speech API (browser native)	Free, no 3rd-party SDK needed
Auth	Flask-Login	Simple session auth, minimal code
CI/CD	GitHub Actions	Free, auto test on every PR
Deploy	Render (free tier)	Auto-deploy on merge to main

⚠ **Browser Compatibility Note:** *Web Speech API has limited support on Safari. Recommend Chrome for full voice features. Safari users will see a graceful fallback message.*

3. Team Structure & Role Rotation

Sprint	Week	Product Owner	Scrum Master	Developers
Sprint 1	Week 3	Yanbin Xu	Yanshuo Liu	Junfan Zhou, Yupei Yang, Ruoqi Hu, Weike Jin, Chenxi Huang
Sprint 2	Week 4	Junfan Zhou	Yupei Yang	Yanbin Xu, Yanshuo Liu, Ruoqi Hu, Weike Jin, Chenxi Huang

Sprint 3	Week 5	Ruoqi Hu	Weike Jin	Yanbin Xu, Yanshuo Liu, Junfan Zhou, Yupei Yang, Chenxi Huang
Sprint 4	Week 6	Chenxi Huang	Yanbin Xu	Yanshuo Liu, Junfan Zhou, Yupei Yang, Ruoqi Hu, Weike Jin

Frontend vs Backend Grouping

Each sprint, members naturally split into frontend and backend focus based on their assigned tasks. One member acts as a bridge (handles API integration between front and back). The bridge person is whoever has the lightest individual task load that sprint.

4. GitHub Collaboration Workflow

4.1 Branch Strategy: Feature Branch Model

- **main** branch: always stable, deployable. Protected by branch rules.
- **Feature branches:** created from main for each task, e.g. **feature/T1.4-daily-words**
- **Bug fix branches:** e.g. **fix/word-bank-delete-error**

4.2 Pull Request (PR) Rules

1. **Every task = one branch = one PR.** Never commit directly to main.
2. **PR description** must include: what was done, how to test it, any notes for reviewer.
3. **At least 1 teammate must Code Review** and approve before merging.
4. **CI must pass (green check)** before merging is allowed.
5. **After merging, delete the remote branch** to keep the repo clean.

4.3 Daily Git Workflow (Every Developer)

Before starting work: git pull origin main → git checkout -b feature/T1.x-task-name

During work: Commit often with clear messages, e.g. "feat: add daily words API endpoint"

When done: git push origin feature/T1.x-task-name → Open PR on GitHub → Tag reviewer

After review: Merge PR → Delete remote branch → Move Trello card to Done

4.4 API Contract: The Key to Parallel Work

On Day 1 of each sprint, the whole team spends 30–60 minutes defining every API endpoint for that sprint. This goes into **docs/api-sprintX.md**. Format:

```
GET  /api/daily-words  → Returns today's 5–8 words with definitions, examples, audio flag
POST /api/word-bank    → { word_id: int } → 201 Created
```

...each endpoint with request params and response JSON structure

Why this matters: Frontend developers can build pages using mock data immediately. Backend developers build APIs to match the contract. When both sides finish, integration is seamless because the interface was agreed upon in advance.

5. Project Structure (Monorepo)

```
academic-english-app/
├── frontend/              # React + Vite
│   ├── src/pages/         # Page components (DailyWords, WordBank, Listening...)
│   ├── src/components/    # Shared UI components (NavBar, AudioPlayer...)
│   ├── src/api/           # API request functions (mock-ready)
│   └── src/assets/        # Images, icons
├── backend/               # Flask
│   ├── app/routes/        # API route handlers
│   ├── app/models/        # SQLAlchemy models
│   ├── app/services/      # Business logic, AI calls
│   └── tests/             # pytest tests
├── docs/                  # API contracts, meeting notes, architecture
├── .github/workflows/     # CI/CD config
├── .env.example           # Environment variable template
└── README.md              # Setup instructions
```

6. Sprint Plans

Sprint 0 (Week 2): Preparation

Activity	Tasks	Assignee	Output
Requirements	Refine user stories, write acceptance criteria	All Team	Updated Backlog
Architecture	System design, DB schema draft, API structure, page routing plan	Yanbin Xu, Yanshuo Liu	Architecture Doc
Environment Setup	Create GitHub repo, setup React+Vite & Flask projects, write README setup guide, configure .env.example	Junfan Zhou, Yupei Yang	Working repo everyone can clone and run

UI/UX Design	Wireframes for all core pages (7 pages: Home, Daily Words, Word Bank, Listening, Speaking, AI Chat, Profile)	Ruoqi Hu, Weike Jin	Figma Prototypes
Sprint 1 Planning	Estimate tasks, define Sprint 1 API contracts, assign work	Chenxi Huang	Sprint 1 Plan

Sprint 1 (Week 3): Vocabulary Module

Sprint Goal: Deliver functional Vocabulary Hub — users can register/login, see daily words, and save to personal word bank.

PO: Yanbin Xu | SM: Yanshuo Liu | Capacity: 7 × 10h = 70h

ID	Task	Pri	Assignee	Hours	Dep	Collaboration Notes
T1.0	Frontend project setup (React+Vite, router, Ant Design, API module with mock)		Yanbin Xu	8	—	Day 1 start. All frontend tasks depend on this.
T1.1	Database schema design & implementation (SQLAlchemy models)		Junfan Zhou	8	—	Day 1 start, parallel with T1.0.
T1.2	Backend Flask API framework + route structure		Yupei Yang	8	T1.1	Before T1.1 merges: setup Flask skeleton, folder structure, config. After T1.1 merges: connect DB and implement route stubs.
T1.3	User authentication (register/login, Flask-Login sessions)		Ruoqi Hu	6	T1.2	While waiting: build login/register frontend pages with mock API. After T1.2 merges: implement auth routes and connect to frontend.
T1.4	Daily Words feature (API + display page + browser TTS)		Weike Jin	8	T1.0, T1.2	While waiting for T1.2: build Daily Words page UI using mock data from T1.0. After T1.2: implement backend API + connect frontend. TTS: use browser SpeechSynthesis API (no backend needed).
T1.5	Word Bank feature (add/remove/list/review)		Chenxi Huang	8	T1.2, T1.3	While waiting: build Word Bank page UI with mock

						data. After T1.3 merges: connect to auth + implement backend CRUD routes.
T1.6	CI/CD pipeline (GitHub Actions: lint + test on PR)		Yanshuo Liu	6	—	Day 1 start, independent. Setup branch protection rules on main.
T1.7	Unit tests for vocabulary APIs		Junfan Zhou	4	T1.4, T1.5	Start after T1.4/T1.5 PRs are up. Test daily-words endpoint, word-bank CRUD, auth guards.
T1.8	Integration testing (end-to-end flow)		Yupei Yang	4	T1.7	Final day: register → login → see words → save to bank → view bank. File bugs as GitHub Issues with clear reproduction steps.

Total planned: 60h / 70h capacity (86% utilization) — leaves buffer for unexpected issues.

Sprint 1 Day-by-Day Coordination

Day	Key Activities	Merge Events
Mon	Sprint Planning (2h): define sprint goal, review tasks, write API contracts. T1.0, T1.1, T1.6 start immediately (no dependencies). T1.2 starts Flask skeleton. Others: build frontend pages w/ mock data.	API contract doc committed
Tue	T1.0 and T1.1 finish and submit PRs. Yupei Yang reviews T1.1 immediately, merges, continues T1.2.	
Wed	T1.2 finishes. T1.3, T1.4, T1.5 fully unblocked. Three devs work in parallel on auth, daily words, word bank.	
Thu	T1.3, T1.4, T1.5 submit PRs. Cross-review. Begin integration: pull all into main, test full flow.	
Fri	T1.7 unit tests + T1.8 integration tests. Bug fixes. Sprint Review + Retrospective.	All tests + fixes merged. Sprint complete.

Sprint 2 (Week 4): Listening + Speaking Basics

Sprint Goal: Deliver Listening Lab (beginner/intermediate) and Speaking Studio Level 1 (pronunciation practice).

PO: Junfan Zhou | SM: Yupei Yang | Capacity: 70h

ID	Task	Pri	Assignee	Hours	Dep	Collaboration Notes
T2.1	Listening content DB structure + seed data		Yanbin Xu	6	—	Day 1 start. Prepare audio clips + transcripts + questions. Seed DB with at least 6 clips across 2 difficulty levels.
T2.2	Audio player component (HTML5 <audio> + custom controls)		Yanshuo Liu	6	—	Day 1 start, independent. Reusable component. Play/pause, speed control, progress bar.
T2.3	Listening comprehension exercise page		Ruoqi Hu	8	T2.2	While waiting: build page layout with mock data. After T2.2: integrate audio player component.
T2.4	AI quiz question generation (OpenAI API)		Weike Jin	8	T2.1	Backend: send transcript to OpenAI, generate comprehension Qs. Return questions in standard JSON format for T2.3 to consume.
T2.5	Word Quiz feature (spelling, listening formats)		Chenxi Huang	6	Sprint 1	Builds on Sprint 1 word bank data. Independent from listening tasks. Can start Day 1.
T2.6	Pronunciation recording (MediaRecorder API + playback)		Junfan Zhou	8	—	Day 1 start, independent. Browser MediaRecorder API. Record user voice, play back, send to backend for analysis.
T2.7	Speech recognition integration (Web Speech API)		Yupei Yang	8	T2.6	Browser SpeechRecognition for real-time transcription. Compare user speech to target sentence for basic scoring.
T2.8	Sentence follow-along practice		Yanbin Xu	4	T2.7	Display sentence, user reads aloud, show match result.

						Lightweight: reuses T2.6 recording + T2.7 recognition.
T2.9	Sprint 1 bug fixes		Ruoqi Hu + Yanshuo Liu	4	—	2 people share bug duty. Check GitHub Issues from Sprint 1. Fix in separate branches, PR as usual.
T2.10	Unit + integration testing		Yanshuo Liu	4	All	Final day: test all listening + speaking flows end-to-end.

Total planned: 62h / 70h capacity (89% utilization).

Sprint 3 (Week 5): Advanced Speaking + AI Conversation

Sprint Goal: Deliver Speaking Studio Level 2 (structured expression with AI feedback) and AI Conversation with 2 guided scenarios (Office Hours + Seminar Discussion).

PO: Ruoqi Hu | SM: Weike Jin | Capacity: 70h

ID	Task	Pri	Assignee	Hours	Dep	Collaboration Notes
T3.1	Structured speaking with topic cards		Yanbin Xu	8	—	Day 1 start. Topic card selection UI + 30-60s recording. Reuses recording component from Sprint 2.
T3.2	AI feedback system (fluency, accuracy, logic scoring)		Yanshuo Liu	8	T3.1	Backend: send user transcript to OpenAI with scoring prompt. Return structured feedback JSON. While waiting: design prompt + test in playground.
T3.3	Academic expression template library		Junfan Zhou	6	—	Day 1 start, independent. Database of useful phrases. Phrases like "I would like to build on that point..."
T3.4	AI conversation dialog engine (2 fixed scenarios)		Yupei Yang	10	—	Day 1 start. Core AI chat backend. System prompts define scenario context + evaluation criteria.
T3.5	Guided scenario frontend + conversation		Chenxi Huang	8	T3.4	While waiting: build chat UI page (message bubbles,

	UI					input). After T3.4: connect to backend dialog engine.
T3.6	Voice input/output for AI chat (Web Speech API)		Ruoqi Hu	8	T3.4	SpeechRecognition for voice input, SpeechSynthesis for AI replies. While waiting: build voice controls UI and test speech APIs standalone.
T3.7	Post-conversation report generation		Weike Jin	5	T3.5	After conversation ends: OpenAI summarizes performance. Display report card with strengths, areas to improve, key phrases used.
T3.8	Sprint 2 bug fixes + integration testing		Junfan Zhou + Yanshuo Liu	6	All	Shared duty. Junfan: bug fixes (3h). Yanshuo: integration test (3h).

Total planned: 59h / 70h capacity (84% utilization) — extra buffer for AI integration complexity.

⚠ **Individual workload check:** Yupei Yang has 10h (at limit). All others ≤ 8h. No one exceeds the 10h/week cap.

Sprint 4 (Week 6): Integration, Polish & Bonus Features

Sprint Goal: Complete system integration, thorough testing, polish, and deliver production-ready product. Bonus features attempted if time allows.

PO: Chenxi Huang | SM: Yanbin Xu | Capacity: 70h

ID	Task	Pri	Assignee	Hours	Dep	Collaboration Notes
T4.1	Progress tracking dashboard		Ruoqi Hu	6	—	Day 1 start. Shows completed exercises, scores, time spent. Pulls data from all modules (vocab, listening, speaking, AI chat).
T4.2	Academic Culture module (text tips + do/don't cards)		Yupei Yang	6	—	Simplified scope: text-based tips cards (no video). Quiz at end using existing quiz component from

						Sprint 2.
T4.3	Advanced listening (detail capture exercises)		Yanshuo Liu	6	—	Moved from Sprint 3. Adds Advanced difficulty level. Reuses existing Listening Lab components.
T4.4	Full system integration testing		Weike Jin	8	All	Test complete user journey across ALL modules. Cross-browser test (Chrome, Firefox, Edge at minimum).
T4.5	User acceptance testing (UAT)		Chenxi Huang	8	T4.4	Test with real user scenarios (the "Sarah" story). Check every acceptance criterion in the backlog.
T4.6	Performance + security optimization		Yanbin Xu	6	—	Page load speed, API response time, input sanitization. HTTPS, CORS config, environment variables for secrets.
T4.7	Bug fixes and refinements		Junfan Zhou + Ruoqi Hu + Yanshuo Liu	10	T4.4	3 people share. Pick bugs from GitHub Issues by severity. Target: zero P1 bugs, all P2 bugs documented.
T4.8	Documentation (user guide + API docs)		Junfan Zhou	4	—	User guide: how to use the app (with screenshots). API docs: update docs/ folder with final endpoint specs.
T4.9	Final demo preparation + rehearsal		All Team	8	—	Prepare demo script, seed demo data, rehearse "Sarah" scenario. Assign speaking parts. Run through twice minimum.
T4.N1	Free Talk AI conversation mode		Yanshuo Liu	8	Sprint 3	Open topic selection, no fixed scenario. Reuses Sprint 3 dialog engine with different system prompt.
T4.N2	Challenge Mode (difficult scenarios)		Junfan Zhou	6	Sprint 3	Disagreement scenarios, rapid follow-up questions. Reuses dialog engine with

						harder prompts.
T4.N3	Difficulty levels for listening (Beginner/Inter/Advanced labels)		Weike Jin	3	T4.3	Add filter/label UI to listening page. Quick win if T4.3 is done early.

Core tasks total: 62h / 70h capacity. Nice-to-have tasks add up to 17h extra — only attempted if Thursday check-in confirms core is stable.

7. Product Backlog (Ordered by Value)

#	ID	User Story	Pri	Pts	Sprint Delivery
1	**US01**	Daily academic words with definitions, examples, audio	Must	5	Sprint 1
2	**US02**	Save words to personal bank for later review	Must	3	Sprint 1
3	**US03**	Listen to lecture clips with comprehension questions	Should	8	Sprint 2
4	**US04**	AI pronunciation feedback on speaking	Should	8	Sprint 2
5	**US12**	Track progress (dashboard with scores, time)	Must	5	Sprint 4
6	**US05**	Vocabulary quizzes (spelling, listening, fill-in-blank)	Must	5	Sprint 2
7	**US06**	Difficulty levels in listening exercises	Must	5	Sprint 4 (T4.3 + T4.N3)
8	**US07**	Structured speaking practice with AI feedback	Should	8	Sprint 3
9	**US08**	AI conversation practice (guided scenarios)	Nice to have	13	Sprint 3
10	**US10**	Academic culture tips and classroom norms	Nice to have	5	Sprint 4 (simplified)
11	**US09**	Free conversation mode	Nice to have	8	Sprint 4 Nice-to-have
12	**US11**	Challenge mode for difficult situations	Nice to have	8	Sprint 4 Nice-to-have

8. Agile Ceremonies

Daily Standup (15 min — text in WeChat/Discord group)

Each member posts 3 lines every morning:

1. What did I complete since last standup?
2. What will I work on next?
3. Any blockers? (tag the person who can unblock you)

SM updates Trello board and burndown chart after reading standups.

Sprint Planning (2 hours, Day 1)

1. PO presents sprint goal
2. **Whole team writes API contracts for this sprint** (30–60 min)
3. Team selects backlog items, breaks into tasks
4. **Individual workload check: no one exceeds 10h**
5. Commitment made, tasks go on Trello board

Sprint Review (1 hour, Last Day)

Demo working software. PO checks acceptance criteria. Collect feedback.

Sprint Retrospective (1 hour, Last Day)

Three questions: What went well? What didn't go well? What will we change? Actions tracked and reviewed next sprint.

9. Definition of Done (DoD)

Criterion	Standard
Code Complete	All code written, follows conventions, merged to main via PR
Code Reviewed	At least 1 team member reviewed and approved the PR
CI Passed	GitHub Actions green check — lint + tests pass
Unit Tested	≥80% coverage, all tests pass
Integration Tested	Works correctly with other modules
UI/UX Verified	Matches Figma design, responsive down to tablet width
Browser Tested	Works on Chrome + Firefox (minimum). Safari voice features may degrade gracefully
Documentation	Code comments, API docs in docs/ folder updated
No Critical Bugs	Zero P1 bugs, P2 bugs documented in GitHub Issues
PO Accepted	Meets acceptance criteria, PO signs off

10. Testing Strategy

Test Type	When	Responsible	Tools
Unit Testing	During development	Each developer	pytest (backend), Jest (frontend)
Integration Testing	End of each sprint	Assigned tester	pytest + Postman
UI Testing	After feature complete	Ruoqi Hu, Weike Jin	Manual + browser DevTools
Cross-Browser	Sprint 4	Weike Jin	Chrome, Firefox, Edge
User Acceptance	Sprint 4	Chenxi Huang	Real user scenarios
Performance	Sprint 4	Yanbin Xu	Lighthouse, browser DevTools

11. Progress Tracking & Transparency

Artifact	Location	Updated By	Frequency
Product Backlog	Trello board	PO	As needed
Sprint Backlog	Trello sprint board	SM	Daily
Burndown Chart	Trello or spreadsheet	SM	After each standup
Task Status	Trello columns (To Do / In Progress / Done)	Each developer	Real-time
API Contracts	docs/api-sprintX.md in GitHub	Whole team	Sprint Day 1
Meeting Notes	docs/meetings/ in GitHub	SM	After each ceremony

12. Risk Management

Risk	Prob	Impact	Mitigation	Owner
OpenAI API integration issues	Med	High	Prototype in Sprint 0, test prompts early, have fallback (hardcoded responses for demo)	Yupei Yang
Web Speech API browser limits	Med	Med	Recommend Chrome. Build graceful fallback message for unsupported browsers	Ruoqi Hu
Team member unavailable	Low	Med	Cross-training via PR reviews, pair programming, shared docs	SM
Scope creep	Med	High	Strict backlog management, PO gatekeeping, Nice-to-have buffer	PO
Underestimated complexity	Med	Med	10–15% time buffer per sprint, daily standup risk flags	SM

Merge conflicts	Med	Low	Small PRs, frequent merges, API contracts reduce overlap	All

13. Team Member Responsibility Matrix

Member	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Yanbin Xu	PO, Frontend Setup (8h)	Listening DB (6h), Follow-along (4h)	Structured Speaking (8h)	SM, Security (6h)
Yanshuo Liu	SM, CI/CD (6h)	Audio Player (6h), Bug fix (2h), Testing (4h)	AI Feedback (8h), Integration (3h)	Advanced Listening (6h), Bug fix (3h)
Junfan Zhou	Database (8h), Unit Tests (4h)	PO idle, Pronunciation (8h)	Expression Library (6h), Bug fix (3h)	Bug fix (4h), Docs (4h)
Yupei Yang	Backend API (8h), Integration (4h)	SM, Speech API (8h)	AI Dialog Engine (10h)	Culture Module (6h)
Ruoqi Hu	Auth System (6h)	Exercise UI (8h), Bug fix (2h)	PO, Voice I/O (8h)	Dashboard (6h), Bug fix (3h)
Weike Jin	Daily Words (8h)	AI Quiz (8h)	SM, Conversation Report (5h)	Integration Test (8h)
Chenxi Huang	Word Bank (8h)	Word Quiz (6h)	Guided Scenarios (8h)	PO, UAT (8h)

⚠ **Rule:** *If any member's total exceeds 10h during Sprint Planning, tasks must be redistributed before committing.*

14. Deliverables Summary

Sprint	Working Software	Documentation	Agile Artifacts
1	Vocabulary Module (Daily Words + Word Bank + Auth)	DB Schema, API Docs, README	Backlog, Sprint 1 Board, Burndown
2	+ Listening Lab + Speaking L1 + Word Quiz	Test Reports	Sprint 2 Board, Retro Notes
3	+ Speaking L2 + AI Guided Conversation (2 scenarios)	Integration Report	Sprint 3 Board, Feedback Log
4	Complete App (+ Dashboard, Culture, Advanced Listening, bonus features if time)	User Guide, Final Report	All Boards, Demo Script

15. Demonstration Checklist

Demo Structure (15 min total):

1. Opening (1 min): Product introduction

2. Problem Statement (1 min): Why this app matters
3. Target Users (1 min): "Meet Sarah, an incoming international student..."
4. Live Demo (5–7 min): Walk through Sarah's journey — login, learn words, listen to lecture, practice speaking, have AI conversation
5. Technical Architecture (2 min): React + Flask + OpenAI, key design decisions
6. Agile Evidence (2 min): Show backlog, sprint boards, burndown charts, retro improvements
7. Testing Evidence (1 min): CI pipeline, test coverage, cross-browser results
8. Conclusion (1 min): Value delivered, known limitations, future improvements

— End of Plan —